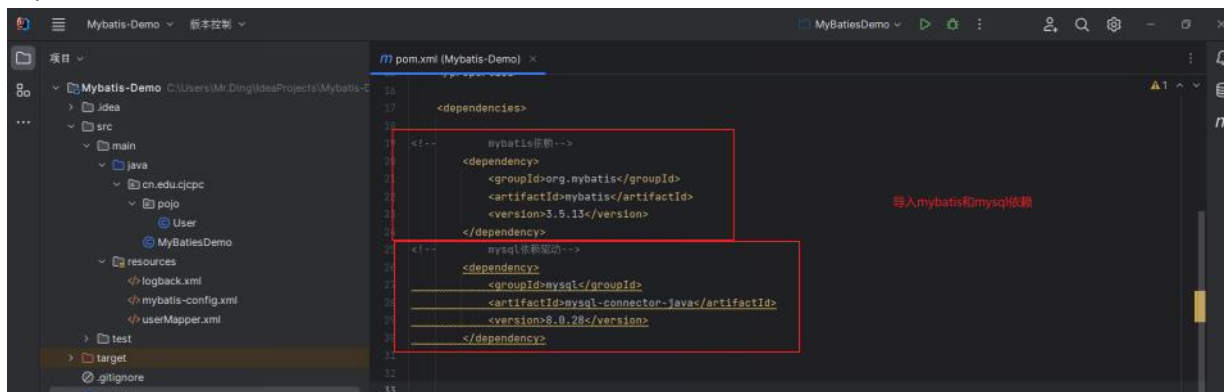


快速入门

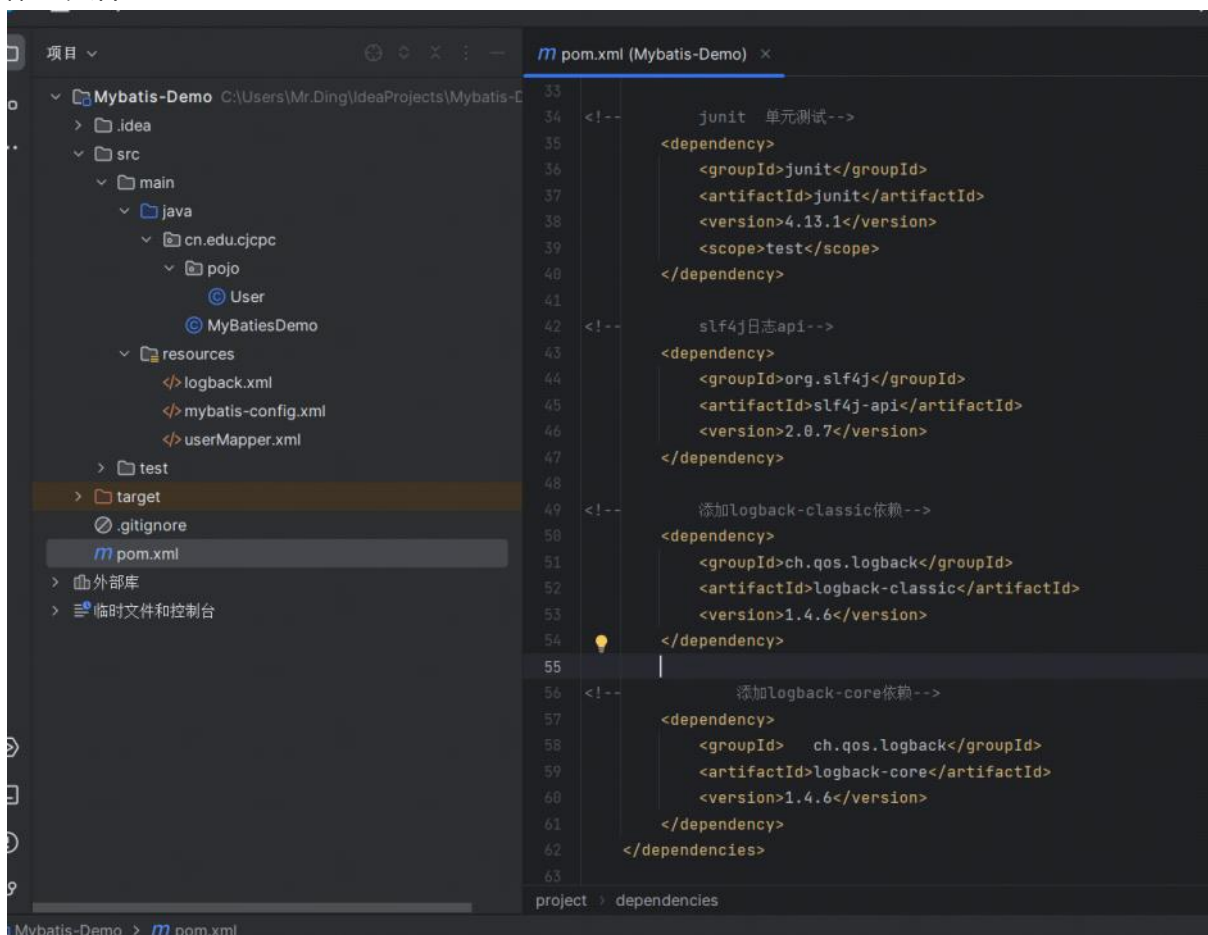
2023年4月24日 10:50

1. 创建user表, 添加数据
2. 创建模块, 导入依赖

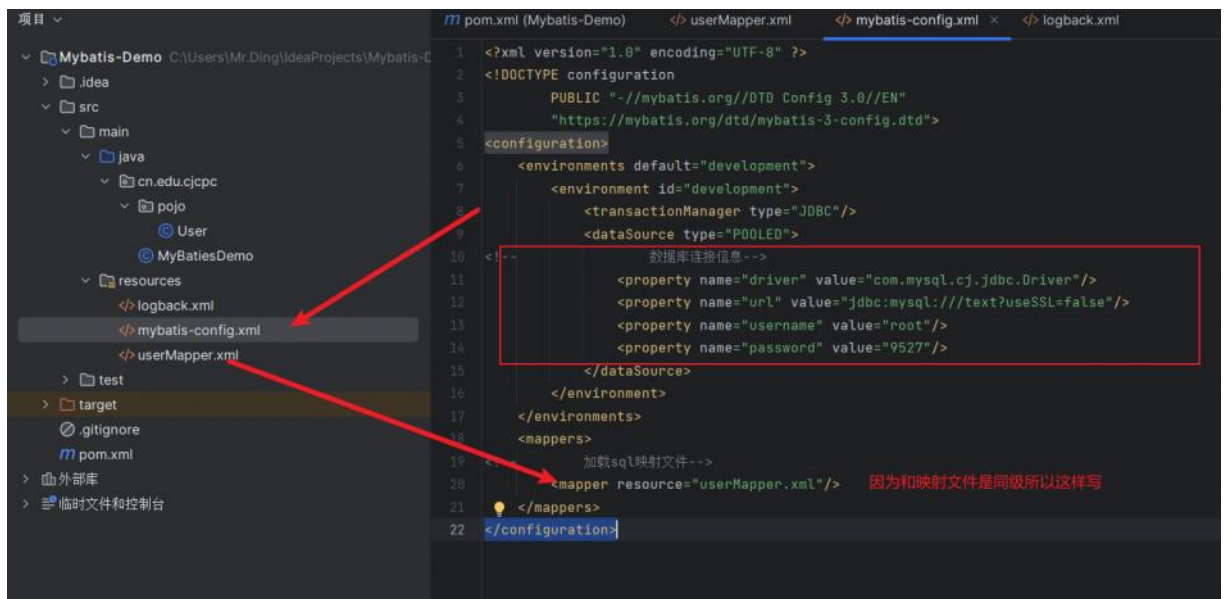
MyBatis官网: [mybatis - MyBatis 3 | 简介](https://mybatis.org/mybatis-3/zh/index.html)



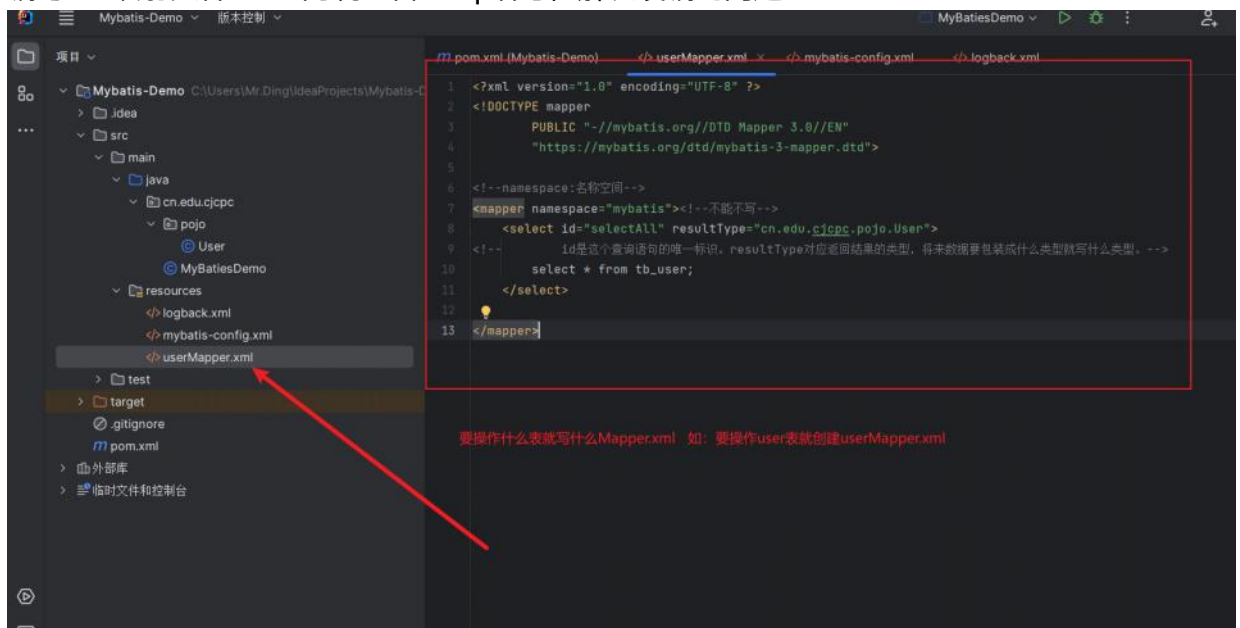
如果要测试要导入Junit依赖, 如果要写日志导入logback依赖, logback还需要导入配置文件



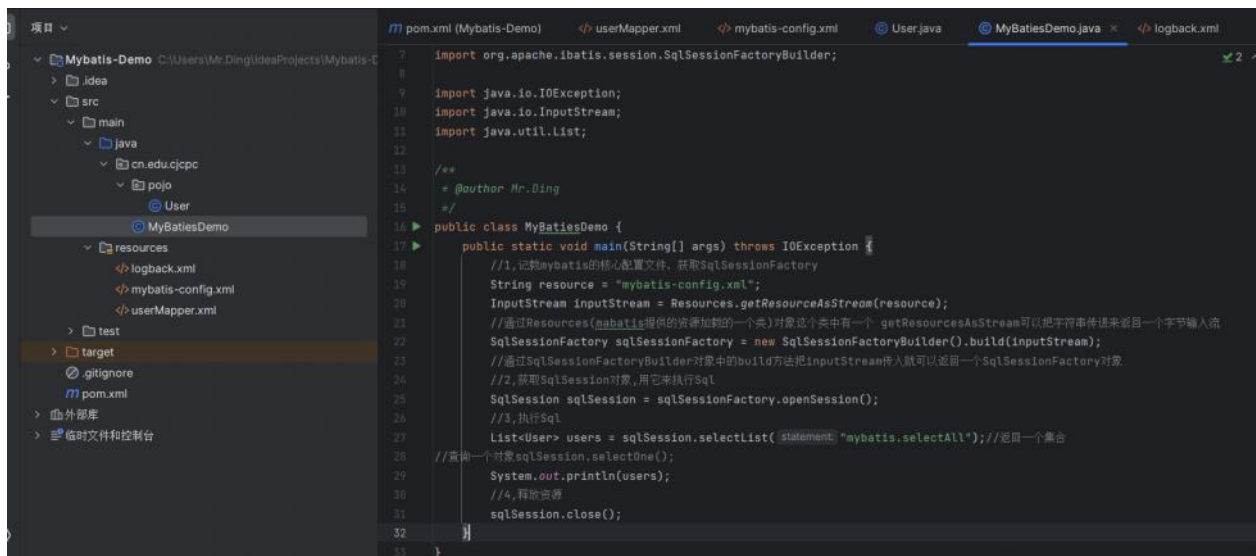
3. 编写MyBatis核心配置文件 就是替换数据库连接信息, 解决硬编码问题



4. 编写SQL映射文件 --> 同统一管理sql语句，解决硬编码问题



5. 编码



1. 定义POJO类

POJO:

"Plain Ordinary Java Object", 简单普通的java对象。主要用来指代那些没有遵循特定的java对象模型, 约定或者框架的对象。

POJO的内在含义是指那些:

有一些private的参数作为对象的属性, 然后针对每一个参数定义get和set方法访问的接口。

没有从任何类继承、也没有实现任何接口, 更没有被其它框架侵入的java对象。

2. 加载核心配置文件, 获取SqlSessionFactory对象
3. 获取SqlSession对象, 执行SQL语句
4. 释放资源

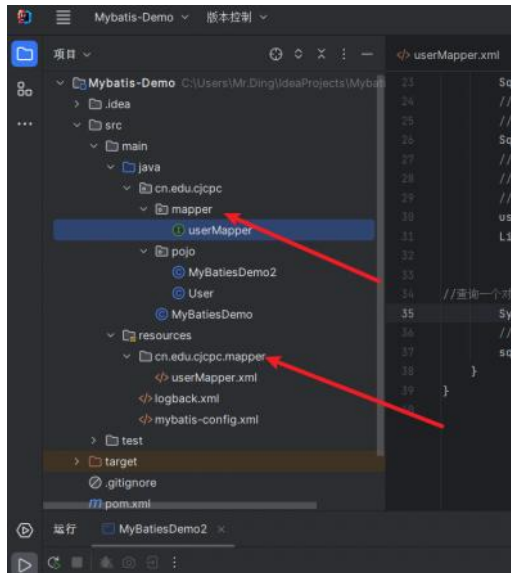
通过Resources(mabatis提供的资源加载的一个类)对象这个类中有一个getResourcesAsStream可以把字符串传进来返回一个字节输入流

通过SqlSessionFactoryBuilder对象中的build方法把inputStream传入就可以返回一个SqlSessionFactory对象

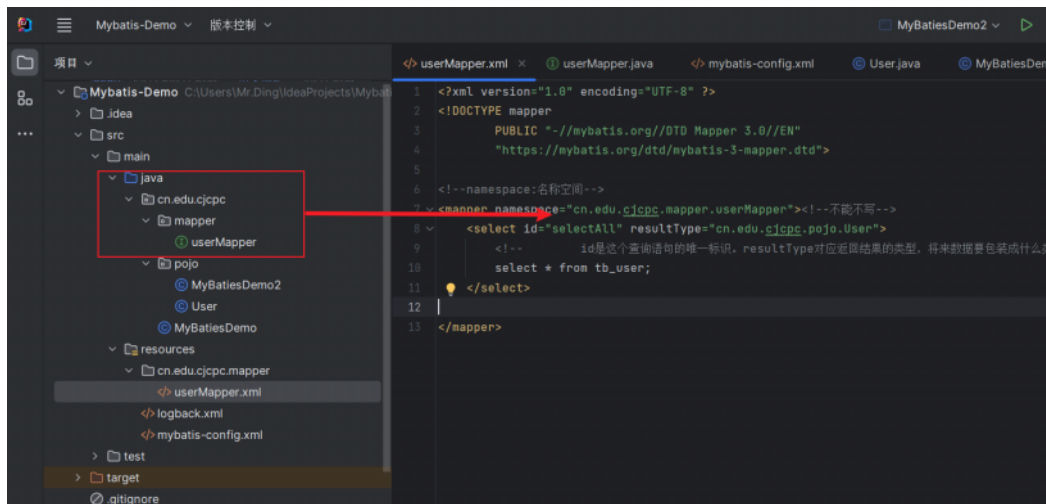
Mapper代理开发

2023年4月25日 19:52

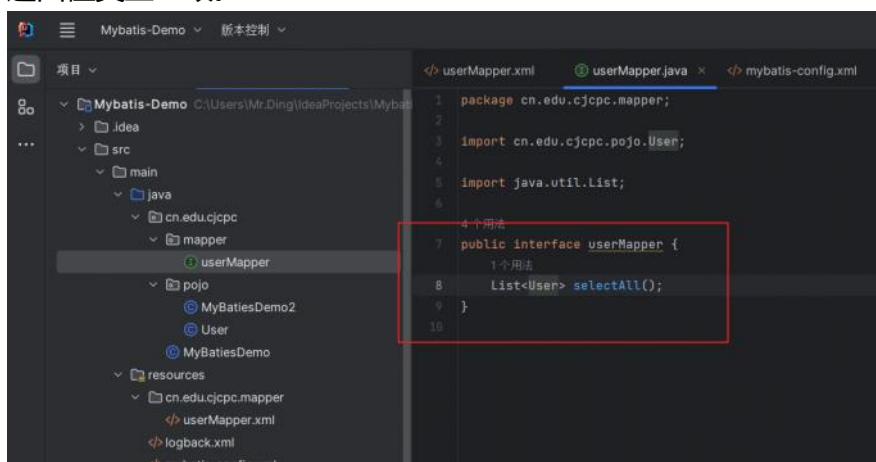
1. 定义与SQL映射文件同名的Mapper接口，并且将Mapper接口和SQL映射文件放在同一目录下



2. 设置SQL映射文件的namespace属性为Mapper接口全限定名



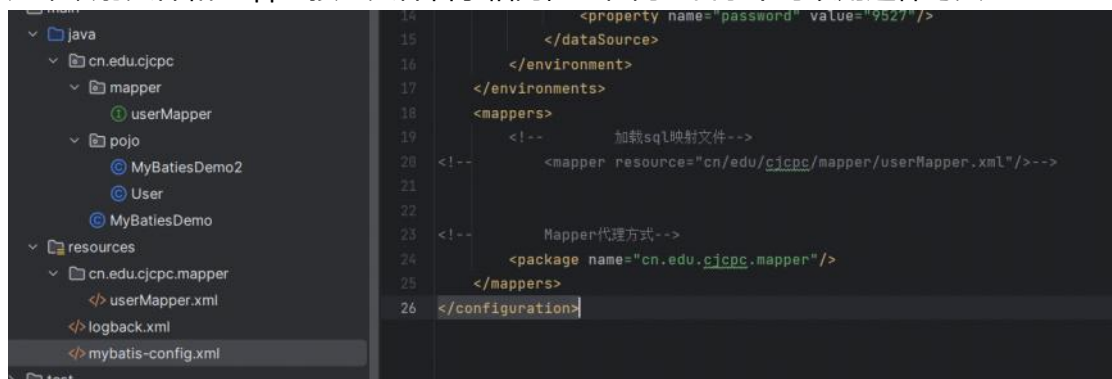
3. 在Mapper接口中定义方法，方法名就是SQL映射文件中sql语句的id，并保持参数类型和返回值类型一致。



4. 编码

1. 通过SqlSession的 getMapper方法获取Mapper接口的代理对象
2. 调用对应方法完成sql的执行

如果映射文件和Mapper接口文件名字相同，且在同一目录下可以用这种写法



散装参数：需要使用@Param("SQL中的参数占位符名称")

如果方法中有多个参数，需要使用@Param

实体类封装参数：只需要保证SQL中的参数名和实体类属性名对应上，即可设置成功

对象的属性名称要和参数占位符名称一致

map集合：只需要保证SQL中的参数名和map集合的键的名称对应上，即可设置成功

```
List<Brand> selectByCondition(@Param("status")int status, @Param("companyName") String  
companyName, @Param("brandName") String brandName);
```

```
List<Brand> selectByCondition(Brand brand);
```

```
List<Brand> selectByCondition(Map map);
```

```
<select id="selectByCondition" resultMap="brandResultMap">  
  select *  
  from tb_brand  
  where  
    status = #{status}  
    and company_name like #{companyName}  
    and brand_name like #{brandName}  
</select>
```


动态查询条件的三种方法

*If

*choose (when, otherwise)

trim (where, set)

foreach

If: 用于判断参数是否有值, 使用test属性进行条件判断

text: 逻辑表达式

```

7
8      <!-- 条件查询-->
9
10     <select id="selectByCondition" resultType="cn.edu.cjcpcc.pojo.Recommand">
11         select * from recommend
12         where
13             <if test="numberone != null and numberone != ''">
14                 numberone like #{numberone}
15             </if>
16             <if test="numbertwo != null and numbertwo != ''">
17                 and numbertwo like #{numbertwo}
18             </if>
19     </select>
20

```

问题:if方法会出现个问题, 如果andSQL语句前的一种条件没有了, 那么这条SQL语句则会出错

例如: select * from recommend where numberone like #{numberone} and like #{numbertwo}, and前的一个条件没有了就会变成 select * from recommend where and numbertwo#{numbertwo} 这样的SQL语句是错误的

解决方法: 恒等式, <where>标签 (这种方式使用的较多)

```

1      <!-- 条件查询-->
2
3      <select id="selectByCondition" resultType="cn.edu.cjcpcc.pojo.Recommand">
4          select * from recommend
5          where 1 = 1
6          <if test="numberone != null and numberone != ''">
7              and numberone like #{numberone}
8          </if>
9          <if test="numbertwo != null and numbertwo != ''">
10             and numbertwo like #{numbertwo}
11          </if>
12      </select>
13

```

在where后加上一个1=1 (本身没有什么作用) 之后的判断条件都添加and即可

<where>标签

```
<!-- 条件查询-->
<select id="selectByCondition" resultType="cn.edu.cjcpcc.pojo.Recommend">
    select * from recommend
    <where>
        <if test="numberone != null and numberone != ''">
            and numberone like #{numberone}
        </if>
        <if test="numbertwo != null and numbertwo != ''">
            and numbertwo like #{numbertwo}
        </if>
    </where>
</select>
```

把were替换为<where> 标签即可

单条件动态查询

- choose (when, otherwise)
- Choose 相当于java中的switch
- When 相当于java中的case
- Otherwise 相当于Java中的default

应用于处理单条件，多结果的查询需求中



具体写法

```
<!-- </select>-->
<select id="selectByConditionSingle" resultType="cn.edu.cjcpcc.pojo.Recommend">
    select * from recommend
    <where>
        <choose>
            <when test="numberone != null and numberone != ''">
                numberone like #{numberone}
            </when>
            <when test="numbertwo != null and numbertwo != ''">
                numberone like #{numberone}
            </when>
        </choose>
    </where>
</select>
```


添加

```

48      <insert id="addPersonal_data">
49          insert into recommend (numberone,numbertwo,image) values ({numberone},{numbertwo},{image})
50      </insert>
51
52
53 </mapper>

```

```

17      //对象参数
18      0 个用法
19      List<Recommend> selectByCondition(Recommend recommend);
20      // List<Recommend> selectByCondition(Map map);
21
22      1 个用法
23      List<Recommend> selectByConditionSingle(Recommend recommend);
24      2 个用法
25      void addPersonal_data(Recommend recommend);
26
27      }

```

测试 已通过: 1共 1 个测试 - 485毫秒

知识点: mybatis的事务问题, .

```

[DEBUG] 18:50:18.638 [main] o.a.i.i.DefaultVFS - Reader entry: <?xml version="1.0" encoding="UTF-8" ?>
[DEBUG] 18:50:18.639 [main] o.a.i.i.ResolverUtil - Checking to see if class cn.edu.cjpc.mapper.RecommendMapper matches criteria [is assignable to (
[DEBUG] 18:50:18.689 [main] o.a.i.t.j.JdbcTransaction - Opening JDBC Connection
[DEBUG] 18:50:18.862 [main] o.a.i.d.p.PooledDataSource - Created connection 1866229258.
[DEBUG] 18:50:18.865 [main] o.a.i.t.j.JdbcTransaction - Setting autocommit to false on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@6f3c660a]
[DEBUG] 18:50:18.873 [main] c.e.c.m.R.addPersonal_data - ==> Preparing: insert into recommend (numberone,numbertwo,image) values (?,?,?)
[DEBUG] 18:50:18.899 [main] c.e.c.m.R.addPersonal_data - ==> Parameters: %1$(String), %2$(String), 图片(String)
[DEBUG] 18:50:18.905 [main] c.e.c.m.R.addPersonal_data - <== Updates: 1
[DEBUG] 18:50:18.905 [main] o.a.i.t.j.JdbcTransaction - Rolling back JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@6f3c660a]
[DEBUG] 18:50:18.906 [main] o.a.i.t.j.JdbcTransaction - Resetting autocommit to true on JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@6f3c660a]
[DEBUG] 18:50:18.906 [main] o.a.i.t.j.JdbcTransaction - Closing JDBC Connection [com.mysql.cj.jdbc.ConnectionImpl@6f3c660a]
[DEBUG] 18:50:18.907 [main] o.a.i.d.p.PooledDataSource - Returned connection 1866229258 to pool.

```

Autocommit 为false 表示关闭了自动提交, 也就是手动提交相当于开启了事务, 运行后没有手动提交, 则下方回滚了事务, 所以数据库中没有添加成功数据。

解决方法: 使用sqlsession.commit()设置自动提交或者把sqlSessionFactory.openSession(true)设置为true

```
62 // map.put("numbertwo", numbertwo);
63 //1.获取SqlSessionFactory
64 String resources = "mybatis-config.xml";
65 InputStream resourceAsStream = Resources.getResourceAsStream(resources);
66 SqlSessionFactory sqlSessionFactory = new SqlSessionFactoryBuilder().build(resourceAsStream);
67 //2.获取SqlSession对象
68 SqlSession sqlSession = sqlSessionFactory.openSession( autoCommit: true);
69 //3.获取Mapper接口的代理对象
70 RecommendMapper recommendMapper = sqlSession.getMapper(RecommendMapper.class);
71 //4.执行方法
72 // List<Recommend> recommends = recommendMapper.selectByCondition(numberone, numbertwo);
73 recommendMapper.addPersonal_data(recommend);
74
75 // sqlSession.commit();
76
77 System.out.println();
78 //5.关闭SqlSession
79 sqlSession.close();
80 }
81 }
```

测试已通过: 1 个测试 ~ 489毫秒

通常，执行插入操作后需要或如插入成功的数据生成的主键值，不同类型数据库获取主键值的方式不同。

在数据库添加成功后，需要获取插入数据库数据的主键的值
比如：添加订单和订单项

在添加语句执行后，相对应的id已经产生可以使用keyProperty和useGeneratedKey将id取出来

keyProperty：将插入或更新操作的返回值赋值给POJO类的某个属性，如果需要设置联合主键，在多个值之间用逗号隔开

useGeneratedKey：该属性会使Mybatis调用JDBC的getGeneratedKeys()方法来获取数据库内生产的主键，如MySQL和SQL Server等自动递增的字段，其默认值为false

动态修改：

```
51
52 <update id="update" >
53     update recommend
54     <set>
55         <if test="numberone != null and numberone != ''">
56             numberone = #{numberone},
57         </if>
58         <if test="numbertwo != null and numbertwo != ''">
59             numbertwo = #{numbertwo},
60         </if>
61         <if test="image != null and image != ''">
62             image = #{image},
63         </if>
64     </set>
65     where id = #{id};
66
67 </update>
68
69 </mapper>
```

使用<set>标签避免当后一条的数据没有时，前一条的SQL语句以逗号结尾的错误，当所有的参数都没有时，<if>标签内的语句都不存在，导致SQL语句变成“update recommend where id = #{id}”的这种错误语句的情况，如果出现不存在系统变量的情况考虑是 <update>标签下的update recommend这条语句没有写。

```

org.apache.ibatis.exceptions.PersistenceException:
### Error updating database. Cause: java.sql.SQLException: Unknown system variable 'numberone'
### The error may exist in cn/edu/cjcpc/mapper/RecommandMapper.xml
### The error may involve defaultParameterMap
### The error occurred while setting parameters
### SQL: SET numberone = ?, numbertwo = ?,
### Cause: java.sql.SQLException: Unknown system variable 'numberone'

at org.apache.ibatis.exceptions.ExceptionFactory.wrapException(ExceptionFactory.java:30)

```

删除:

删除单条数据:

```

69
70 <delete id="deleteById">
71     delete from recommand where id = #{id};
72 </delete>
73

```

```

2 个用法
void deleteById(int id);

```

动态删除:

```

<delete id="deleteByIds">
    delete from recommand where id in
    <foreach collection="ids" item="id" separator="," open="(" close=")">
        #{id}
    </foreach>
    ;
</delete>

```

```

27
28 void deleteByIds(@Param("ids") int[] ids);
29
30
31 }

```

id的个数根据id数组变化而变化

Mybatis 提供了foreach来遍历数组，collection指定要遍历的数组，item指的是遍历出的每一个元素，separator属性指的是以什么为间隔，open属性指的是在开头以什么插入，close属性指的是结尾以什么插入，简化了 SQL语句中 in () 关键字的括号

建议：将来都使用@Param注解来修改Map集合种默认的键名，并使用修改后的名称来获取值，这样可读性更高！

黑马程序员
www.itheima.com

MyBatis 参数传递

MyBatis 接口方法中可以接收各种各样的参数，MyBatis底层对于这些参数进行不同的封装处理方式

➤ 单个参数：

1. POJO类型：直接使用，实体类属性名 和 参数占位符名称 一致
2. Map集合：直接使用，键名 和 参数占位符名称 一致
3. Collection：封装为Map集合
4. List：封装为Map集合
5. Array：封装为Map集合
6. 其他类型：直接使用

➤ 多个参数：封装为Map集合

```
map.put("collection",collection集合);
map.put("arg0",collection集合);
```

```
map.put("collection", list集合);
map.put("list",list集合);
map.put("arg0", list集合);
```

```
map.put("array", 数组);
map.put("arg0", 数组);
```

```
map.put("arg0", 参数值1);
map.put("param1", 参数值1);
map.put("arg1", 参数值2);
map.put("param2", 参数值2);
```

建议：将来都使用@Param注解来修改Map集合中默认的键名，并使用修改后的名称来获取值，这样可读性更高！

[13-参数传递 哔哩哔哩 bilibili](#)关于参数传递的源码讲解，理解的还不是很透彻

正东方向 男人手里 路旁 大约5里左右

分区 添，改，删 的第 12 页

Mybatis-Plus

2023年8月10日 22:47

使用步骤



步骤

SpringBoot整合MyBatis-Plus

②：定义数据层接口与映射配置，继承**BaseMapper**

```
@Mapper
public interface UserDao extends BaseMapper<User> {
}
```

使用plus需要继承BaseMapper 需要指定对应的泛型，该泛型就是实体类

- 继承BaseMapper并指定泛型

```
@Mapper
public interface BookDao extends BaseMapper<Book> {
}
```

开启驼峰命名的配置，需要在配置文件中进行配置

```
1 mybatis:
2   configuration:
3     #开启驼峰命名
4     map-underscore-to-camel-case: true
```

去除数据表头的下划线命名方式

```
10 # Mybatis Plus
11 mybatis-plus:
12   global-config:
13     db-config:
14       table-underline: false # 设置表明不使用下划线方式
15   configuration:
16     map-underscore-to-camel-case: false # 设置字段不使用下划线方式
17
```

开启plus日志

数据层开发

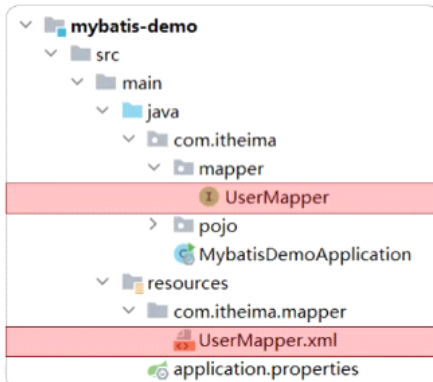
- 为方便调试可以开启MyBatisPlus的日志

```
mybatis-plus:
  configuration:
    log-impl: org.apache.ibatis.logging.stdout.StdoutImpl
```


映射配置

2026年3月15日 15:59

- 在Mybatis中，既可以通过注解配置SQL语句，也可以通过XML配置文件配置SQL语句。
- 默认规则：
 1. XML映射文件的名称与Mapper接口名称一致，并且将XML映射文件和Mapper接口放置在相同包下（同包同名）。
 2. XML映射文件的namespace属性为Mapper接口全限定名一致。
 3. XML映射文件中sql语句的id与Mapper接口中的方法名一致，并保持返回类型一致。



```
@Mapper
public interface UserMapper {
    public List<User> findAll();
}
```

Mapper接口

```
<mapper namespace="com.itheima.mapper.UserMapper">
    <select id="findAll" resultType="com.itheima.pojo.User">
        select id, username, password, name, age from user
    </select>
</mapper>
```

XML映射文件

注解式的mapper写法和xml的写法都要用，注解式用来写一些简单的sql语句，不要拘泥于一种方式

```
#指定xml映射配置文件的位置
mysql.mapper-locations=classpath:mapper/*.xml
```

Mybatis中封装查询结果，什么时候用 `resultType`，什么时候用`resultMap`?

- 如果查询返回的字段名与实体的属性名可以直接对应上，用`resultType`。
- 如果查询返回的字段名与实体的属性名对应不上，或实体属性比较复杂，可以通过`resultMap`手动封装。

```

</select>

<resultMap id="empResultMap" type="com.dyx.pojo.Emp">
    <id property="id" column="id"/>
    <result property="username" column="username"/>
    <result property="password" column="password"/>
    <result property="name" column="name"/>
    <result property="gender" column="gender"/>
    <result property="phone" column="phone"/>
    <result property="job" column="job"/>
    <result property="salary" column="salary"/>
    <result property="image" column="image"/>
    <result property="entryDate" column="entry_date"/>
    <result property="deptId" column="dept_id"/>
    <result property="createTime" column="create_time"/>
    <result property="updateTime" column="update_time"/>

    <collection property="exprList" ofType="com.dyx.pojo.EmpExpr">
        <id property="id" column="ee_id"/>
        <result property="empId" column="ee_empid"/>
        <result property="begin" column="ee_begin"/>
        <result property="end" column="ee_end"/>
        <result property="company" column="ee_company"/>
        <result property="job" column="ee_job"/>
    </collection>
</resultMap>

<!-- 根据id查询员工基本信息以及员工工作经历信息-->
<select id="getById" resultMap="empResultMap">
    select
        e.*,
        ee.id ee_id,
        ee.emp_id ee_empid,
        ee.begin ee_begin,
        ee.end ee_end,
        ee.company ee_company,
        ee.job ee_job
    from emp e left join emp_expr ee on e.id = ee.emp_id
    where e.id = #{id}
</select>

```

总结，在一对一时查询语句可以直接使用`resultType`，一对多时使用`resultMap`，并将返回的数据一一对应。